

System Design Notes: Sloppy Quorum

1. The Background: Why do we need it?

In distributed databases, data is copied (replicated) across multiple servers (nodes) so that if one crashes, your data is still safe.

Normally, systems use a **Strict Quorum**. This means before a "write" is considered successful, a strict majority of the nodes responsible for that data must acknowledge it.

- **The Problem:** If a network goes down or servers crash, you might not reach that majority. A strict quorum system will simply **block the write** and return an error.
- **The Goal:** Some systems (like shopping carts) must *always* accept writes, even during failures. We need a way to keep accepting data when the primary nodes are offline.

2. What is a Sloppy Quorum?

A **Sloppy Quorum** is a fallback mechanism that prioritizes **Availability** over strict consistency.

If the exact nodes that are *supposed* to store your data are unreachable, the system will temporarily write your data to **other healthy nodes** in the cluster—even if those nodes don't normally handle that specific data.

Think of it like this:

If the mailman tries to deliver a package to your house (the primary node) but you aren't home, instead of returning the package to the post office (failing the write), they leave it with your neighbor (a healthy, random node) to hold onto for you.

3. How It Works (The Scenario)

Let's say your database cluster has 5 nodes: **A, B, C, D, and E**.

Your data is configured to be stored on nodes **A, B, and C**.

1. **Node A crashes.**
2. A user tries to write new data. The system needs 3 nodes to confirm the write to meet its quota.
3. Because A is down, it can only reach B and C.

4. Instead of failing, the system "gets sloppy." It walks down the server ring and picks the next healthy server—let's say, **Node D**.
5. The system writes the data to **B, C, and D**. The write succeeds!

4. The Magic Trick: Hinted Handoff

Sloppy Quorum is always paired with a cleanup process called **Hinted Handoff**.

When Node D accepted that temporary data, it attached a digital sticky note (a "hint") to it that says: *"Hey, this actually belongs to Node A."*

- Node D keeps pinging Node A in the background.
- Once Node A reboots and comes back online, Node D "hands off" the data back to Node A.
- Node D then deletes its temporary copy.
- The system is back to normal.

5. The Trade-Offs (CAP Theorem)

Sloppy Quorum is a classic example of choosing **Availability over Consistency** (AP in the CAP theorem).

Pros	Cons
High Availability: The database almost never rejects a write, even during massive network outages.	Stale Reads: Because data was written to a "neighbor" node, a user might try to read that data before the handoff happens, getting an older version of the data.
Fault Tolerance: System repairs itself automatically once nodes come back online.	Temporary Inconsistency: It takes time for the data to eventually make its way to the correct home (Eventual Consistency).

6. Where is it used?

You will see Sloppy Quorums and Hinted Handoffs in highly scalable, decentralized NoSQL databases designed by big tech companies:

- **Amazon DynamoDB** (Where the concept was popularized)
- **Apache Cassandra**
- **Riak**